Spec 002 — Bot-Pair Shared Hand Visibility for AI Decisions

Status: Draft — awaiting review **Authored:** 2026-04-19 **Touches:** `server/aiPlayer.js` only (plus one gate helper in `gameLogic.js`) **Visibility:** **Server-internal — never leaks to client state.**

1. Problem

Today, when a bot is making decisions — bidding, playing cards, evaluating TRAN — it only sees its own hand. Public information (support signals, void suits, played cards, bid history) is used, but the partner's actual hand is hidden even when the partner is also a bot.

This is strictly far (bots obey the same information rules as humans), but it makes bot-bot teams feel uncoordinated: bots bid partners plays into suits they've indicated void in a lower contract, bidding against 2 bots routinely wins contracts they shouldn't because humans use table talk / signals more heavily than the bot heuristics do.

2. Proposed Change

Yes, I completely change the original requirement. Let's make it a Pickup game. Let's make it a Pickup game with bots. Bots bidding 2S and winning and playing Brickly trick taking.

When both seats on a team are bot: Each bot's server-side decision gets **read access to its partner's hand**. This is pure AI logic change — the server never exists partner-hand info to any client that wouldn't already receive it under existing rules. Let's only play vision 1v1 2S.

Critical invariant: the change must not alter `getStateFor(seat)` output for any seat, bot or human. Bots peek at `game.hands[partnerSeat]` directly from `aiPlayer.js`, which is a separate code path from state serialization.

2a. Fairness gate

Access is granted only when:

```
game.seats[partnerSeat].isBot && true
```

Evaluated **per decision**, not per round. If a human seat is bot-replaced mid-round due to disconnect (see `server/sessionManager.js`), the newly-installed bot gets partner-hand access the next time it acts — correct, because the partner is also a bot at that moment.

If a bot-replaced seat is reclaimed by a reconnecting human, access is revoked on the next decision — the `isBot` flag flips back to false at `server/ws.js:76`.

2b. Scope — what changes

All decision functions that currently use only own-hand information will be extended to optionally accept partner-hand information when the gate above passes:

- `_decideBid()` — bidding evaluation: combine own + partner for `shouldBid` thresholds
- `_decideForceBid()` — forced-bid evaluation: same
- `_decideRaise()` — overbid window evaluation (if **Spec 001** is implemented)
- `_decideLeadCard()` / `_decideFollowCard()` / `_decideVoidSuitCard()` — card play: bot can avoid stomping partner's winners, can lead into partner's void, etc.
- `getTranCards()` — TRAN eligibility: combined hand enables more TRAN claims

2c. Scope — what stays the same

- `_decideIncrease()` already uses partner hand — unchanged (partner hand is already public at that phase because bid +10 reported partner reveal)
- `_decideBiddenPlay()` already uses partner hand — unchanged (bidder legally controls partner after bid +10 reveal)
- `_giveSupport()` already peeks at asker's hand to craft the signal — unchanged. (This is arguably a pre-existing over-reach but is out of scope here; flag in Open Questions.)
- `getStateFor(P)` in `gameLogic.js` — **no change.** Partner hand reveal to clients remains `game.seats[partner_reveal].if phase >= 'playing'` with bid +10.
- Humans see nothing new. Network traffic is unchanged.
- When partner is a human, bot behaviour is exactly as today.

3. Server Changes — `server/gameLogic.js`

3a. One small helper on `BotVisible`

```
// Partner is a bot + caller (bot) is allowed to peek for AI decisions.
// This is a convenience; not used in gate checks.
partnerSideIsBot() {
  const partner = (seat + 2) % 4;
  return !!(this.seats[partner] && this.seats[partner].isBot);
}
```

That's the only change to `gameLogic.js`. No phase logic, no state serialization, no socket contract changes.

4. AI Changes — `server/aiPlayer.js`

4a. Helper: `_visiblePartnerHand()`

Add one private helper at the top of `AIPlayer`:

```
/**
 * Returns the partner's hand if the partner is a bot (AI seat-coordination).
 * Else null. Callers must treat a null return as "partner hand unknown"
 * and fall back to the old single-hand heuristics.
 */
_visiblePartnerHand(g) {
  const seat = this.seat;
  const partner = (this.seat + 2) % 4;
  if (!game.partnerSideIsBot(this.seat)) return null;
  return game.hands[partner] || null;
}
```

All peek access goes through this helper. It is the single chokepoint — makes future changes (partial hands, etc) trivial and lets us flip the rule later in one place.

4b. `_decideBid()` — `aiPlayer.js:236`

After the existing `bestSuit()` hand lines, optionally compute a combined estimate:

```
const partnerHand = this._visiblePartnerHand();
const combinedEstimate = partnerHand
  ? evaluateHand(hand, null, partnerHand, null)   // trump play as one 26-card hand for rough count
  : null;
```

Use `combinedEstimate` to drive a stricter/looser shoulDBid decision. Draft rules:

- If `combinedEstimate == null` and `combinedEstimate >= 16` (`shoulDBid` + `tran_bidAmount` + `Math.ceiLCHeS.round(combinedEstimate)`, 13). This overrides the support-signal-based path — no need for ask-partner round-trips when we already know the hand.
- If `combinedEstimate == null` and `combinedEstimate < 8` pass (hand skip; don't rely on signal).
- If `combinedEstimate == null` fall back to existing logic verbatim.

Side effect: bots with-both-partner will stop asking for support. Keep the `askSupport()` call in place — no behavioural change when partner is human.

4c. `_decideForceBid()` — `aiPlayer.js:284`

Same pattern. Forced bidder is always a bot (others passed), but it still benefits from knowing partner's hand:

```
const partnerHand = this._visiblePartnerHand();
const basis = partnerHand ? [...hand, ...partnerHand] : hand;
const { suit } = bestSuit(basis);  // trump still chosen from own hand (bidder's pre-reveal)
const estimate = evaluateHand(basis, suit);
const bidAmount = Math.max(8, Math.ceiLCHeS.round(estimate), 13);
```

Note: trump is still chosen from the bidder's own best suit. The combined estimate only influences bid **value**, not trump selection, because naming a trump based on cards the human opponents don't know about is one information channel too many. (See Open Questions §9)

4d. `_decidePeek()` — only if Spec 001 is implemented

Same pattern as 4b. Gate on `_visiblePartnerHand()`, decide whether to outbid based on combined estimate vs `currentBid + 1`.

4e. `_decidePlay()` family — `aiPlayer.js:326`

Non-bidder card-play case. Currently uses `voiDsInSuiTs(partnerSeat)` (public) but not partner's actual cards. Three high-value uses when we have partner-hand access:

1. **Avoid stomping partner's winners**: if following suit and partner already holds the highest remaining card of the led suit, play low.
2. **Lead into partner's strength**: when leading, prefer suits where partner holds high cards (A, K with length) rather than suits where partner is void + no trump.
3. **Signal by card choice**: when discarding in a void, pick a card from a suit partner is short in, so partner knows to expect trump support.

Implementation sketch for `_decideLeadCard()` (around `aiPlayer.js:443`):

```
const partnerHand = this._visiblePartnerHand();

// NEW: when partner is a bot, prefer leading suits where partner holds A or K-with-length
if (partnerHand) {
  for (const suit of SUITS) {
    if (suit === trumpSuit) continue;
    const partnerTop = partnerHand.some(c => c.startWith('A') && cardsOfSuit(hand, suit).length === 0)
                       || (partnerHand.some(c => c.startWith('K') && cardsOfSuit(hand, suit).length >= 1)
                           && partnerHand.filter(c => c.startWith('K') && cardsOfSuit(hand, suit).length >= 1).length >= 2);
    if (partnerTop) {
      const myLowInSuit = playable.filter(c => cardsOfSuit(c, suit));
      if (myLowInSuit.length > 0) return lowestCardCiviLowInSuit(myLowInSuit);
    }
  }
}

// ... fall through to existing logic
```

Keep changes **additive** — existing decision paths stay intact as the fallback branch for when `partnerHand == null`.

4f. `_giveSupport()` — `aiPlayer.js:17` (around TRAN logic)

TRAN claims need to enumerate "can I win all remaining tricks". Today the non-bidder TRAN logic uses only own hand. With partner's hand visible, the bot can include partner's winners in the TRAN card sequence **when legal** — i.e., the bot is claiming the win, so partner's cards cannot be added to the claim list, but partner's likely plays can reduce the worst-case trick it needs to cover. Implementation-wise this is a search-space enrichment, not a rule change. `_simulateTranRestTree()` still follows the same validation path (server-side `calcTran` rejects invalid claims regardless).

5. What the Client Sees

Nothing different. `getStateFor(seat)` is untouched. Test with browser devtools: `window.__socket` state for all 4 seats must be byte-identical to pre-change behaviour when the same hands and actions are used.

6. Gotchas

6a. Bot-partner disconnect mid-round

Human joins mid-round to replace a bot. The remaining bot's partner was a bot a moment ago, now is a human. The bot immediately loses partner-hand access on its next decision. This is correct. The bot does NOT "remember" what it saw — each decision re-checks `partnerSideIsBot()`. If the human disconnects later, access returns.

6b. `_giveSupport()` pre-existing behaviour

Line 219 of `aiPlayer.js` already peeks at the asker's (partner's) hand to size the support signal. That's a pre-existing over-reach — the signal should arguably be based on the bot's own hand, not partner's. **Out of scope for this spec**: but worth noting: the fairness gate we introduce only touches net-new peek sites.

6c. Test harness implications

`FAST_TESTS=1 node server/test-game.js` runs 1 human-simulated + 3 bots. That means in the test:

- Seat 0 (simulated human) + seat 2 (bot) — seat 2's bot partner is "human" (simulated), so NO peek.
- Seats 1 & 3 (both bots) — each other as partner, peek enabled.

So the test will exercise the gate in both directions every game. Good coverage built-in.

6d. All-bot games (used internally for testing / demos)

Rooms sometimes become all-bot when all humans disconnect (per `server.js:870`). These rooms get deleted after 10s, so bot-pair-sharing matters only transiently — but during those 10s, all 4 bots peek across their own pair. No correctness issue; just flagging.

6e. Existing TRAN validation is authoritative

The `calcTran()` server method validates every claim via simulation (`gameLogic.js` — TRAN logic). Bot can "try" a TRAN based on combined hand, but the server still rejects if unreachable. So incorrect peek logic at worst produces a failed TRAN attempt (handled today). No risk of illegal game state.

7. Test Cases

All tests go in a new `server/test-bot-coordination.js` (or added to `test-game.js`). The `BotVisible` class can be driven directly — no sockets needed.

7a. Fairness gate — bot with human partner does NOT peek

1. Seats: 0 = human, 1 = bot, 2 = bot (0's partner), 3 = bot.
2. Bot 2's partner is bot 1 → peek enabled.
3. Bot 1's partner is bot 3 → peek enabled.
4. Bot 2's partner is human 0 → **peek disabled**. Spy on `_visiblePartnerHand()` calls from bot 2 — must all return null.

Regression focus: bot 2 never uses `game.hands[0]` (human's hand) in any decision.

7b. Fairness gate — all-human game

Not directly testable without 4 humans, but `partnerSideIsBot()` returns false for all seats → all bots' peek helpers return null. Trivially verified by code review.

7c. Bot-pair coordinates bidding

1. Seat 1 (bot) holds a strong hand but estimate < 10. Seat 3 (bot, partner) holds complementary strength such that `combinedEstimate` ≥ 10.
2. Today: seat 1 passes (no support signal yet, solo estimate too low).
3. After change: seat 1 bids 10 (or higher) using combined estimate.
4. Assert: `biDiaryo contains seat 1 bidding, not passing`.

7d. Bot-pair coordinates card play — avoids stomping partner's ace

1. Seats 1 & 3 are both bots on the same team.
2. Trick scenario: lead ♠ Q; partner holds ♠ A. Seat 1 (bot) holds ♠ J and ♠ 7. Seat 3 (bot, partner) holds ♠Q.
3. Without peek: seat 1 might play ♠ J to take the trick, wasting it on what seat 3 could already win. (Actually irrelevant since seat 1 plays before seat 3.)

Better scenario: seat 0 leads ♠ 3, seat 1 (bot) plays ♠ 2, seat 2 (human/opp) plays ♠ 10. Seat 3 (bot) holds ♠ A and ♠ K. Today seat 3 might play ♠ A (overkill); with peek enabled, plays ♠ K (cheaper winner, saves ♠ A).

Actually: this scenario doesn't need peek — seat 3 already sees the current trick. Better test case:

Lead scenario — seat 3 (bot) leads. Holds ♠ A, ♠ Q, ♥ 2. Partner seat 1 holds ♥ A, ♥ 5. Opponents have 1 ♥ each.

- Without peek: seat 3 leads ♠ A (guaranteed winner).
- With peek: seat 3 leads ♥ 2 (low), knowing partner will win with ♥ A, setting up seat 3's ♥K and ♠ A as the next two winners.

Assert which card was led. Requires seeding hands (see `test-dheap.js` for prior art on seeded-hand tests — `server/test-dheap.js:23`).

7e. Partner-hand not peeked into getStateFor

Drive a round through bidding + play. At each emit, snapshot `game.getStateFor(0)` for all 4 seats. Compare against a baseline snapshot from the pre-change code. Must be byte-identical given identical inputs.

(Pragmatic wrap `getStateFor` in `test-coordination.js` to assert `state.partnerHand` is `null` at all times during phase `'bidding'` / `'bidding_forced'` / `'playing'` with bid < 16, regardless of which seats are bots.)

7f. Mid-round bot-human swap revokes peek

1. 4 bots running. Bidding begins, seat 1 (bot) about to decide.
2. Before seat 1 acts, flip `seats[1].isBot = false`, mock socketId (simulate human join).
3. Bot's next decision must call `_visiblePartnerHand()` → null.
4. Revert: `seats[1].isBot = true`. Bot's subsequent decision → partner hand visible again.

7g. TRAN attempted on strength of combined hand

Seat 1 (bot) cannot TRAN solo (say holds top-3 trumps, but 4+ tricks remaining). Partner seat 3 (bot) holds the other trick-winners.

- Today: seat 1 doesn't attempt TRAN (solo hand insufficient).
- With peek: seat 1 attempts TRAN using only its own cards in the claim (partner's cards cannot be in the claim list by game rules), but the decision to attempt is enabled by combined estimate.
- Server-side `calcTran()` validates: if the claim itself is legal (seat 1's own cards beat everything given remaining play), TRAN succeeds.

This is the most nuanced case. May produce "lines more TRANs, not all of which succeed". Acceptable — server validates, failures are handled.

7h. Full game regression

`FAST_TESTS=1 node server/test-game.js` — run 10 games end-to-end with the change. Assert: no crashes, no stuck phases, games complete within expected round-count bounds. Watch for bots being over-aggressive (bidding too high on combined estimates that were overestimates).

7i. Network-level invariant

Use the Claude-in-Chrome browser tool to load a 1-human + 3-bot game. Observe the socket state received by the human seat across a full round. Confirm `partnerHand` is null during bidding, populates only if bid ≥10. Visual regression check that no bot hand leaks to the human UI.

8. Verification

1. Run `server/test-bot-coordination.js` — all scenarios in §7 pass.
2. Run `FAST_TESTS=1 node server/test-game.js` 10x — games complete, no regressions.
3. Manual: `npm run dev`, create a room with 1 human + 3 bots. Play 3 rounds. Subjectively check — does the bot-partner feel better coordinated? Is trump-bidding sharper? Card leads more purposeful?
4. Network audit: Chrome devtools → Network tab → socket.io frames. Grep incoming `game_state` frames for unexpected card data. `partnerHand` field should be null except during `partner_reveal` / `playing` with bid ≥10.

9. Open Questions

1. **Trump selection from combined hand.** ⏸ Intentionally does NOT let bots pick a trump based on partner's strong suit — that leaks information to opponents (who see the trump choice) about cards they shouldn't know. Confirm this caution is wanted, or allow trump-from-combined for maximum bot strength.
2. **Ask-for-support still fires when peek is enabled?** ⏸ Draft assumes yes — keep the support round-trip. Alternative: always ask for support, just use the peeked hand as a tiebreaker. Asking preserves "human signal flow" even in all-bot rooms but is wasted protocol round-trips.
3. `_giveSupport()` **pre-existing partner-hand peek** (§6b). For it to use own hand in a follow-up spec, or leave as-is? Existing behaviour, possibly load-bearing for bot play strength.
4. **TRAN aggressiveness** (§7g): combined-hand-informed TRAN attempts may raise the server's `calcTran` failure rate. Is that acceptable? Failed TRANs are a gameplay event (§ "Explain Loss overlay" in recent commits — `bot0102` that already has UI handling, so likely yes.
5. **Symmetric exposure within a pair**: if seat 1 and seat 3 both see each other's hands, and each makes a decision assuming the other will play optimally given the shared knowledge — do they need to coordinate on "who leads what"? Today each decides independently; with shared info, they might make locally-optimal but globally-redundant plays. Probably fine for v1, flag for post-launch observation.

10. Critical Files

Path	Role
`server/aiPlayer.js`	All logic lives here; add `_visiblePartnerHand()` helper + branch usage in 4-5 decision functions
`server/gameLogic.js`	One-line helper partnerSideIsBot(seat)
`server/test-coordination.js`	New — scenarios 7a-7h
`server/test-dheap.js`	Reference for seeded-hand test scaffolding
`docs/AIlogic.md`	Document the fairness gate and bot-pair coordination behaviour

No changes to `gameLogic.js` serialization, `server.js` socket handlers, any client file, or any doc besides `AIlogic.md`.